Abbozzo Simulatore

March 5, 2026

0.1 Assunzioni

- Viene modellizzato un sistema parts-to-picker: i pod mobili conteneti varie SKU ciascuno vengono portati a delle workstation fisse dai robot della fleet. Le SKU sono distribuite su vari pod.
- Non è considerato il numero di una data SKU su un pod (e quindi neanche dei vincoli di inventario o il task di replenishment), pertanto neanche il volume delle SKU ha importanza

1 Input Information

Sezione ispirata da [MXL18], articolo di presentazione del software RAWSim-O.

Il simulatore avrà bisogno di alcune informazione iniziali al partire dalle quali creare l'istanza di simulazione. La sezione è impostata seguendo quanto fatto da [MXL18] adattandolo al nostro caso semplificato.

1.1 Layout specification

Un primo input riguarda il layout fisico del sistema.
A questo livello ci sono vari parametri da fissare:

- P = numero di pod
- R = numero di robot
- W = numero di workstation
- [BBE17] introduce anche un parametro $\xi \in \{0.005, 0.05, 0.2\}$ che indica la frazione di SKU per pod. In questo senso ogni pod contiene un numero di SKU compreso in $[1, \lceil \xi P \rceil]$.

Una prima struttura dati necessaria è un array che contiene le info riguardanti le SKU contenute in un pod. Potrebbe essere conveniete al posto di avere una matrice pensare ad una biezione tra l'id del pod e la sua posizione nella griglia.
Può anche essere utile precalcolarsi le distanze di ciascun pod dalle workstation.

1.2 Scenario Specification

In questa fase vengono specificati dei parametri che permettono la generazione degli ordini.

Alcuni, tra cui [JLH23] e [JP23] e il simulatore RAWSim-O stesso, seguono la logica di generazione degli ordini presentata da [BBE17]. Per ogni ordine si genera inizialmente il numero di item da una distribuzione uniforme discreta in $[1, 0.1 * R + 1]$ (per lui R non è il numero di pod ma un valore inferiore che indica quanti pod sono necessari a soddisfare gli ordini, ed è fissato $R \in [5; 100]$) e poi vengono estratte le SKU. Ogni SKU è caratterizzata da un parametro di priorità (estratto da un esponenziale di parametro $\lambda = 0.5$) che determina la probabilità che venga estratta

$$p_i = \frac{\text{parametro di priorità SKU i}}{\text{somma dei parametri di priorità di tutte le SKU}}$$

.

1.3 Controller Configuration

Regola la parte di in che modo gestire l'ottimizzazione.

2 Struttura della Simulazione

Prima di entrare nei dettagli tecnici della simulazione, descrivo brevemente la dinamica del sistema.

Gli ordini arrivano in modo continuo (per semplicità assumeremo tempi di interarrivo costanti) e vanno ad inserirsi nel **backlog degli ordini**. A intervalli regolari viene eseguito un ottimizzatore che, considerando solo un sottoinsieme di ordini presenti nel backlog, determina l'assegnazione ordini-workstation-tempo e la sequenza con cui i pod devono visitare le diverse workstation.

La simulazione deve inoltre contenere un **emulatore del movimento dei pod**, ilcui compito è di prendere lo scheduling generato dall'ottimizzatore e simulare i movimenti dei robot incaricati di trasportare i pod verso le workstation assegnate.

Si noti che l'ottimizzatore attinge allo stato dell'emulatore per determinare i vincoli, mentre l'emulatore attinge all'output dell'ottimizzazione per definire i task.

2.1 Stato del simulatore esterno

Il simulatore esterno dovrà tracciare le seguenti informazioni:

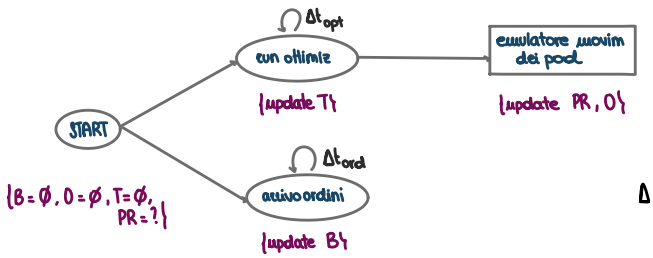
1. **Backlog degli ordini**: struttura in cui vengono memorizzati gli ordini entrati nel sistema e non ancora considerati nell'ottimizzazione.

2. **Stato degli ordini**: struttura dati in cui si va a specificare lo stato degli oridni già considerati dall'ottimizzatore. Questo dato è necessario sia all'ottimizzatore (alcuni vincoli verranno da qui, ad esempio per considerare gli oirdini già aperti alle workstation) sia all'emulatore che ne aggiornerà anche il valore.

3. **Posizione dei robot**: struttura dati che è sia input che output dell'emulatore.

4. **Coda dei task**: questa struttura è output dell'ottimizzatore e input dell'emulatore.

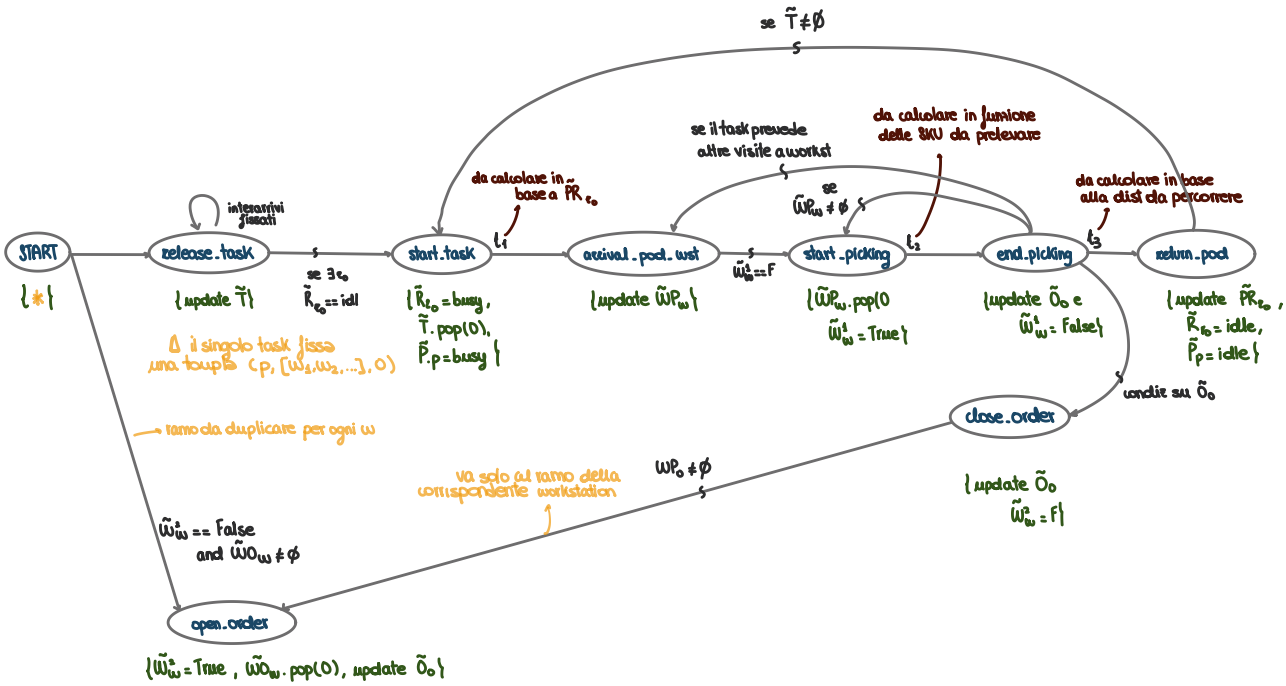
STATO: (B = backlog ordini , O = stato ordini già considerati nell'opt prec , PR = posizione robot , T = coda task)



Δ run ottimiz sfrutta le info in B e O
emulatore movim dei pod sfrutta le info in T e PR e O

emulatore movim dei pod

STATO = ($\tilde{PR}$ = posizione robot , $\tilde{T}$ = coda dei task released , $\tilde{O}_o$ = info sul singolo ordine , $\tilde{P}_p$ = stato pod p , $\tilde{R}_r$ = stato robot r , $\tilde{W}^1_w$ = se c'è un'azione di picking in w , $\tilde{W}^2_w$ = se c'è un ordine aperto in w , $\tilde{WP}_w$ = coda dei pod alla workstation w , $\tilde{WO}_w$ = coda degli ordini alla workstation w)



* $\tilde{T} = \emptyset$, $\tilde{PR} = PR$, $\tilde{P}_p$ = idle $\forall p$, $\tilde{R}_r$ = idle $\forall r$, $\tilde{O}_o$ = { aperto se $o \in O$ / in coda se $o \notin O$, $\tilde{WP}_w = \emptyset$ $\forall w$, $\tilde{WO}_w$ da inizializzare in base ai risultati dell'opt , $\tilde{W}^1_w$ = False $\forall w$, $\tilde{W}^2_w$ dipende da O

2.2 Emulatore del movimento dei robot

Questa è il modulo che a partire dallo scheduling ottenuto simula la parte di movimentazione dei pod. Immagino che l'ottimizzatore consideri variabili decisionali del tipo

$$\text{ordini-workstation-tempo} \quad x_{owt} = \begin{cases} 1 & \text{se allo step } t \text{ l'ordine } o \text{ è aperto alla workstation } w \\ 0 & \text{altrimenti} \end{cases}$$

$$\text{pod-workstation-tempo} \quad y_{iopwt} = \begin{cases} 1 & \text{se la SKU } i \text{ per l'ordine } o \text{ è già stata prelevata dal pod } p \text{ alla workstation } w \text{ allo ste} \\ 0 & \text{altrimenti} \end{cases}$$

(due delle variabili decisionali del modello matematico della Barnhart [BJS24]).

2.2.1 Come definisco lo stato?

Ad ogni istante devo tenere traccia di:

- **Stato degli Ordini**: per ogni ordine devo sapere il suo stato (*open*, *waiting* o *closed*) e le SKU mancanti per il suo completamento
- **Stato dei Pod**: per ogni pod devo sapere se è in giro o al suo posto (*busy* o *idle*)
- **Stato dei Robot**: per ogni robot devo sapere il suo stato (*idle* o *busy*) e la sua posizione se *idle*
- **Stato delle Workstation**: per ogni workstation devo sapere la coda degli ordini (determinata da x_{owt}), se c'è un ordine attualmente *open* (*empty* o *occupied*), se c'è un'azione di picking già in corso (*idle* o *busy*) e la coda dei pod in attesa di processamento
- **Coda dei Task Rilasciati**: coda prioritaria che ha come risorsa la flotta dei robot. All'inizio di ogni simulazione performata dall'emulatore questa coda sarà vuota, verrà man mano riempita quando i task dalla *coda dei task* verrano rilasciati.

Un oggetto che dovrà essere definito è il `task` che dovrà contenere le seguenti informazioni: (1) il pod da prelevare, (2) la lista ordinata di workstation da visitare, (3) per ogni workstation visitata bisogna specificare quante SKU sono da prelevare e per quale ordine, (4) un parametro di priorità che mi dice quando è necessario che avvenga il task[1]. Tutte queste info sono deducibili da x_{owt} e y_{iopwt}, e averle tutte assieme è vantaggioso per quando bisognerà aggiornare lo stato dell'emulatore.

2.2.2 Quali sono gli eventi?

- `release_task`: eventi che si schedulano all'inizio dell'emulatore a istanti di tempo dati dal parametro di priorità dei task. Questo evento prende il task e lo aggiunge alla coda dei task rilasciati.
- `start_task` e `return_pod` sono due eventi che indicano, rispettivamente, l'inizio e la fine di un task. Verranno quindi aggiornati gli stati relativi al pod e al robot coinvolto.
- `arrival_pod_wst`, `start_picking` e `end_picking`[2] sono gli eventi relativi alla movimentazione del pod. Questi eventi vanno ad aggiornare lo stato delle workstation e degli ordini coinvolti.
- `open_order` e `close_order` sono i due eventi che regolano quali ordini sono in servizio nelle workstation.

La relazione tra questi eventi è meglio visibile dal diagramma inserito nella pagine precedente.
L'evento `start_task` è triggerato dalla presenza di un released task non ancora allocato e dalla presenza di almeno un robot in stato *idle*, si può quindi inserire un blochetto di ottimizzazione che permette di scegliere il robot a cui allocare il task.[3]

[1] Mi viene da pensare che possa essere comodo un trade-off tra quando il pod deve essere alla workstation secondo la soluzione dell'ottimizzatore e la distanza tra il pod e la workstation + deve dipendere da che ordini ho aperti nelle workstation?

[2] Ho separato i due eventi in modo da poter considerare un tempo di picking:[BBE17] considera questo tempo costante, mentre [Zhe+23] introduce una funzione del numero di SKU da prelevare e la loro dimensione.

[3] Ottimizzo con policy "assegno il task al robot *idle* più vicino".

2.2.3 Quali sono le code del sistema?

Nel sistema ci sono principalmente 3 code:

1. **Coda degli ordini alle workstation**: ogni workstation ha una sua coda degli ordini la cui priorità è determinata da x_{owt}.

2. **Coda dei pod alle workstation**: ogni workstation ha una sua coda dei pod la cui priorità è determinata da FIFO.

3. **Coda dei task rilasciati**: in questa coda sono presenti i task già rilasciati ordinati secondo il loro fattore di priorità.

3 Domande al prof

1. Io sto pensando alla simulazione come costruita a due livelli: il primo si occupa della generazione degli ordini e dell'ottimizzazione periodica, mentre il secondoe simula gli spostamenti dei pod. E questi due livelli devono in qualche modo interagire in quanto (1) lo stato attuale del sistema influenza alcuni vincoli dell'ottimizzatore (ad esempio se non ho finito di servire l'ordine 1 alla workstation 2 devo inserire un vincolo per cui $x_1, 2, 0 = 1$), e viceversa (2) l'output dell'ottimizzatore è l'input della simulazione dei movimenti.
   Per farli interagire senza troppi problemi, posso assumere che l'emulatore simuli per un tempo pari a Δt_{opt} in modo che mentre gira l'ottimizzatore il sistema sia di fatto "congelato" e poi faccio ripartire l'emulatore resettando la coda dei task rilasciati? Bisogna solo scegliere se i task già allocati li si porti a termine oppure no (per avere poi gli input giusti sulla posizione dei robot e sullo stato degli ordini). Alternativamente, potrei assumere (mi sembra più drastico) che l'ottimizzatore parta solo quando ho finito di servire tutti gli ordini considerati nell'ottimizzazione precedente.

2. Se considero le variabili y_{pwt} come valide indicazioni di scheduling dei pod, in quale istante della simulazione devo effettivamente attivare la richiesta di movimentazione del pod verso la workstation?
   Mi vengono in mente due approcci: (1) impostare una coda delle richieste che attinge ai robot come risorsa (problema: come decido chi ha la priorità tra pod necessari allo stesso step) oppure (2) generare richieste "anticipate" che tengano conto di quanto disti il pod dalla workstation.
   Seconda questione: Se ho dei robot liberi può avere senso che inizio a prelevare pod che mi serviranno? Più che altro in un caso un po' limite in cui ho molti più robot che workstation, potrebbe essere problematico allocare tutti i robot perché implicherebbe molto movimento (non tutto necessario nell'immediato) di pod attorno alle workstation.

3. Qual'è la planimetria del warehouse?

4 Questioni a cui devo pensare

1. Cosa faccio nel caso in cui $y_{p,w_1,t} = 1$ e $y_{p,w_2,t+1} = 1$? Due approcci: (1) creo il task incodando le due workstation, (2) riporto il pod alla sua posizione e poi alla seconda workstation.

2. PROBLEMA: se $y_{p,w_1,t} = 1$ e $y_{p,w_2,t+2} = 1$ ha effettivamente senso che il pod torni al suo posto? C'è qualcosa nell'ottimizzatore che si assicuri che ciò non accada?

References

[BJS24] Cynthia Barnhart, Alexandre Jacquillat, and Alexandria Schmid. "Robotic warehousing operations: a learn-then-optimize approach to large-scale neighborhood search". In: *ArXiv* abs/2408.16890 (2024). URL: https://api.semanticscholar.org/CorpusID:272310402.

[BBE17] Nils Boysen, Dirk Briskorn, and Simon Emde. "Parts-to-picker based order processing in a rack-moving mobile robots environment". In: *European Journal of Operational Research* 262.2 (2017), pp. 550–562. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2017.03.053.

[JLH23] Guoshuai Jiao, Haobin Li, and Min Huang. "Online joint optimization of pick order assignment and pick pod selection in robotic mobile fulfillment systems". In: *Computers & Industrial Engineering* 175 (2023), p. 108856. ISSN: 0360-8352. DOI: 10.1016/j.cie.2022.108856.

[JP23] Jan-Erik Justkowiak and Erwin Pesch. "A column generation driven heuristic for order-scheduling and rack-sequencing in robotic mobile fulfillment systems". In: *Omega* 120 (2023), p. 102897. ISSN: 0305-0483. DOI: 10.1016/j.omega.2023.102897.

[MXL18] Marius Merschformann, Lin Xie, and Hanyi Li. "RAWSim-O: A Simulation Framework for Robotic Mobile Fulfillment Systems". In: *Logistics Research* 11.8 (2018). ISSN: 1865-035X. DOI: 10.23773/2018_8.

[Zhe+23] Lu Zhen et al. "How to Deploy Robotic Mobile Fulfillment Systems". In: *Transportation Science* (Sept. 2023). DOI: 10.1287/trsc.2022.0265.